

UNIT-II

→ Disjoint Sets - disjoint set operations

A set is a collection of elements. Two sets are disjoint if they have no common elements.

Eg:-

- Students according to majors
- Students according to GPA

It supports the following operations

- 1) make(i):- This operation is used to construct the set $\{i\}$.
- 2) Find(i):- This operation is used to find the 'name' of the set to which it belongs. The name returned is the item 'i' which is at the root of the tree
- 3) Union(i, j):- This operation takes the name of two sets and combines them into one.

UNION Operation:-

UNION(i, j) means the elements of set i , and elements of set j are combined. If we want to represent UNION operation in the form of tree, then ' i ' is the parent and ' j ' is the child.

Algorithm U(i, j)

```
{
    int i, j;
    PARENT(j) = i;
}
```

Eg:- Find Union($1, 3$), Union($2, 5$) and then union($1, 2$)

initially PARENT array contains zeros

0	0	0	0	0	0
1	2	3	4	5	6

UNION($1, 3$)

PARENT(3) = 1

0	0	1	0	0	0
1	2	3	4	5	6



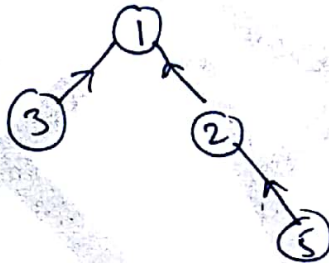
UNION (2,5)

PARENT (5) = 2

0	0	1	0	2	0
1	2	3	4	5	6



UNION (1,2)



PARENT (2) = 1

0	1	1	0	2	0
1	2	3	4	5	6

Find operation:-

Find(i), implies that it finds the root node of i^{th} node, in other words it returns the name of the set i . In union(1,3), FIND(1)=1 and FIND(3)=1 since its parent is 1.

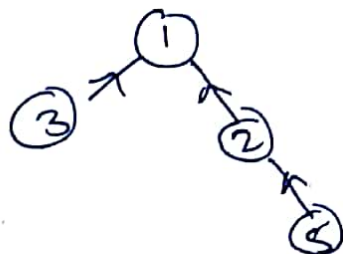
Algorithm F(i)

```

{
  integer i, j;
  j = i;
  while PARENT(j) > 0
    j = PARENT(j);
  return(j);
}

```

Eg:- Find 5 for the following tree



0	1	1	0	2	0
1	2	3	4	5	6



FIND(5), $i=5$, $j=i$, $j=5$

(PARENT(5) = 2 > 0) Condition true

$j = \text{PARENT}(5)$ i.e. $j=2$

(PARENT(2) = 1 > 0) Condition true

So ' j ' = PARENT(2) i.e. $j=1$

(PARENT(1) = 0 > 0) is false

Return(j). So, 1 is the root node of node 5

weighting rule for UNION (i, j):-

If the number of nodes in tree ' i ' is less than the number in tree ' j ', then make ' j ' the parent of ' i '.
otherwise make ' i ' the parent of ' j '.

Algorithm UNION(i, j)

// Union sets with roots i and j , $i \neq j$, using the weighting rule
// $PARENT(i) = -COUNT(i)$ and $PARENT(j) = -COUNT(j)$

{

integer $i, j, x;$

$x = PARENT(i) + PARENT(j);$

if ($PARENT(i) > PARENT(j)$)

then

{

$PARENT(i) = j;$ // i has fewer nodes

$PARENT(j) = x;$

}

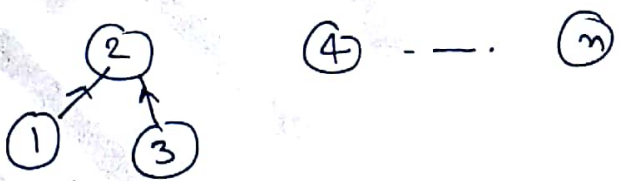
}



UNION(1, 2) =



UNION (2, 3) number of nodes in tree 2 is 2,
number of nodes in tree 3 is 1, so $2 > 1$
make parent as 2 and 3 as child.



Collapsing Rule

If 'j' is a node on the path from i to its root then set $PARENT(j) = root(i)$. A more sophisticated algorithm of FIND using collapsing rule is given below.

Algorithm FIND(i)

// Find the root of the tree containing element i.

// Use the collapsing rule to collapse all nodes

from i to root i

{

j = i;

while $PARENT(j) > 0$

j = $PARENT(j)$;

k = i;

while $k \neq j$ // collapse nodes from i to root j

{

t = $PARENT(k)$;

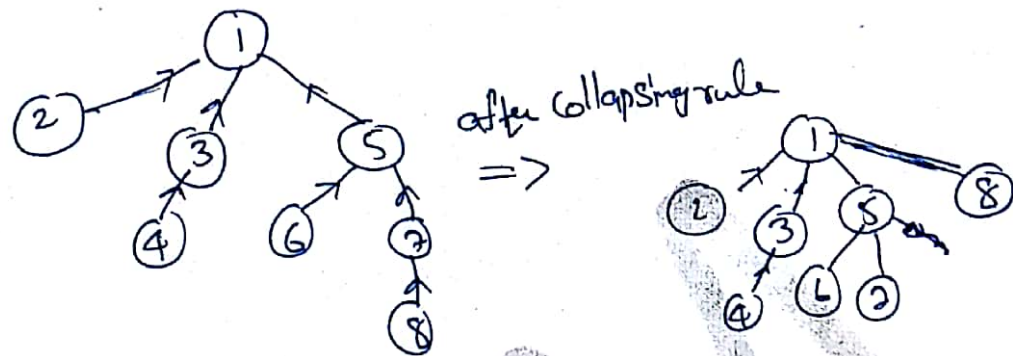
$PARENT(k) = j$;

k = t;

}

return (j);

}



Next will apply 8 Find operations for above tree as follows

Find(8) Find(8) Find(8) Find(8)
Find(8) Find(8) Find(8) Find(8)

- Find(8) = root node i.e. 1
- By using first Find algorithm, it will require 3 moves upward to reach the root node.

For 8 Find operations $8 \times 3 = 24$ moves

- If we use Find algo. using collapsing rule, the first Find(8) requires going up 3 link fields and then resetting 3 links.
- Each of remaining 7 finds requires going up only 1 move, the total cost is now only 10 moves

AND/OR Graphs

The AND/OR graph is a directed graph whose root node represents the main problem while rest of the nodes represents the sub-problem. And the directed edges represents the decomposition relationship existing between problems and subproblems.

The nodes of AND/OR graph are categorized into two groups based on the type of problem decomposition.

They are

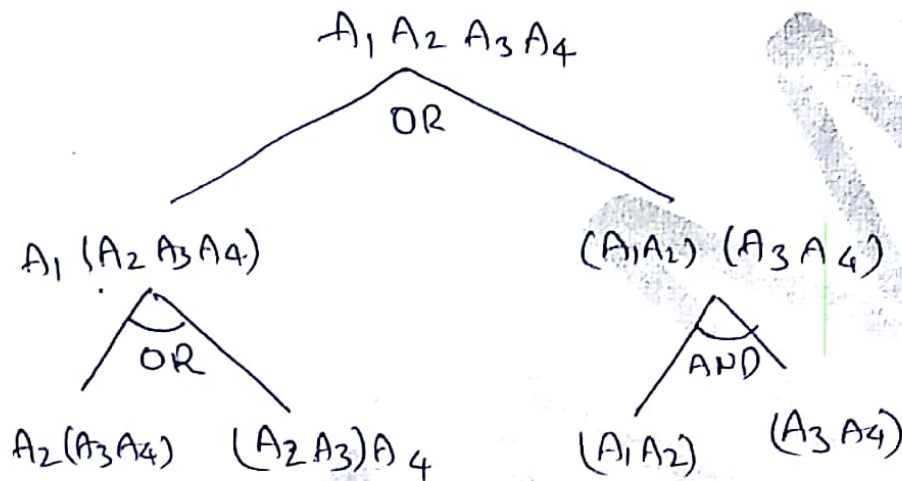
1) AND Nodes:-

A node whose solution can be obtained by solving all its successors is known as AND node

2) OR Nodes:-

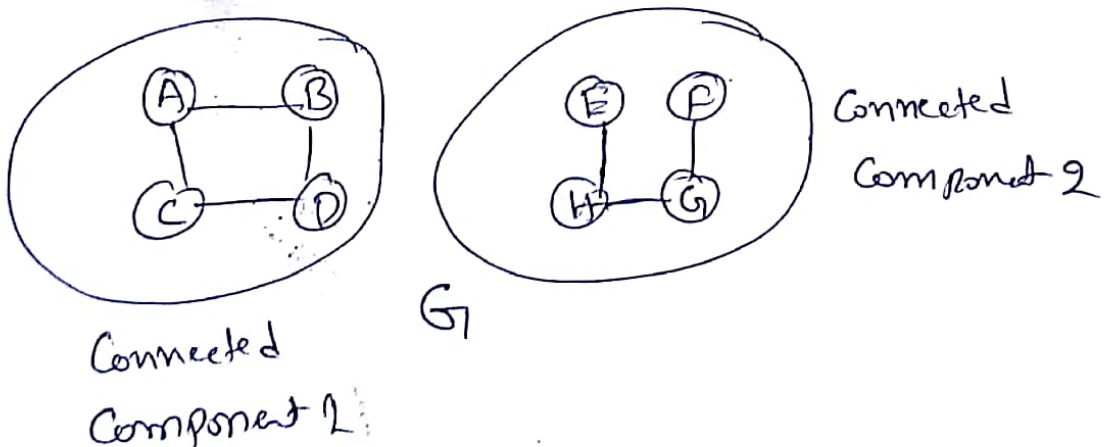
A node whose solution can be obtained by solving any one of its successor is known as OR node

Eg:-



Connected Components:-

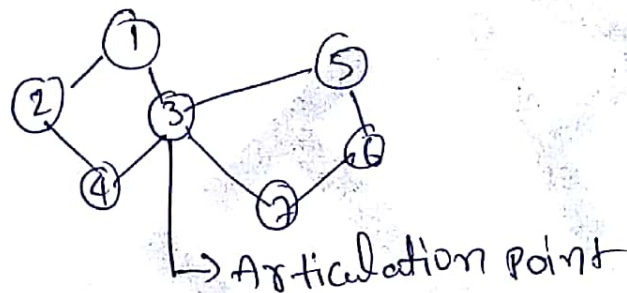
If graph is not connected then its maximal connected subgraphs are called Connected Components of graph



Articulation point:-

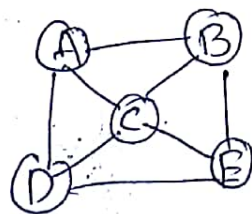
~~It is a point~~

Articulation point is direct vertex in the graph the removal of which partitions the graph into 2 or more non empty components.



Biconnected Component:-

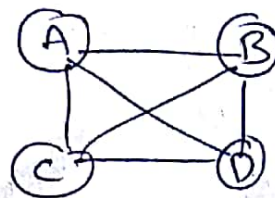
A graph is said to be biconnected if it doesn't contain any articulation point



Spanning Tree:-

A spanning tree is a subgraph of 'G', that contains all the vertices (n) of G and $n-1$ edges.

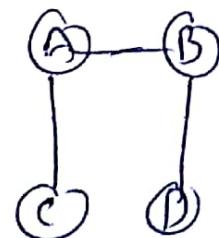
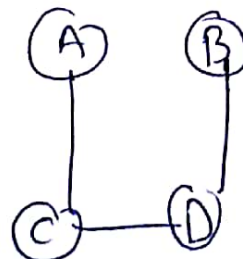
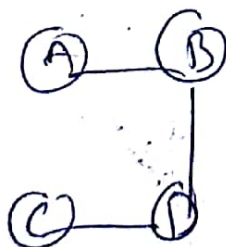
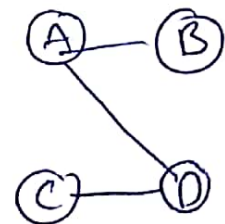
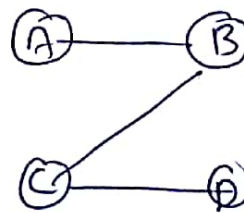
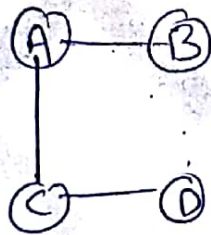
Eg:-



$G(V, E)$

$$n = |V| = 4$$

Spanning trees of above graph are





Back Tracking:-

Back Tracking is a method of determining the correct solution to some computational problem by examining all the available paths. In this method if a particular path leads to an unsuccessful solution then its previous path is examined inside to find the correct solution to the problem.

Applications

- n Queen problem
- Sum of subsets
- Hamiltonian Cycle
- Graph coloring

8- Queen problem:-

A classic Combinatorial problem is to place 8-queens on a 8×8 chessboard in such a way that no two queen can attack each other i.e. no two queens are placed on the same row, column or diagonal.

1	2	3	4	5	6	7	8
		Q ₁					/
					Q ₂		
	Q ₃						
						Q ₄	
Q ₅							
			Q ₆				
							Q ₇
				Q ₈			



Sum of subsets

For a given set of n objects with weights $(w_1, w_2, \dots, w_n)$ and positive integer M , it is required to find a subset S' of the given set S , such that

$$\rightarrow S' \subset S$$

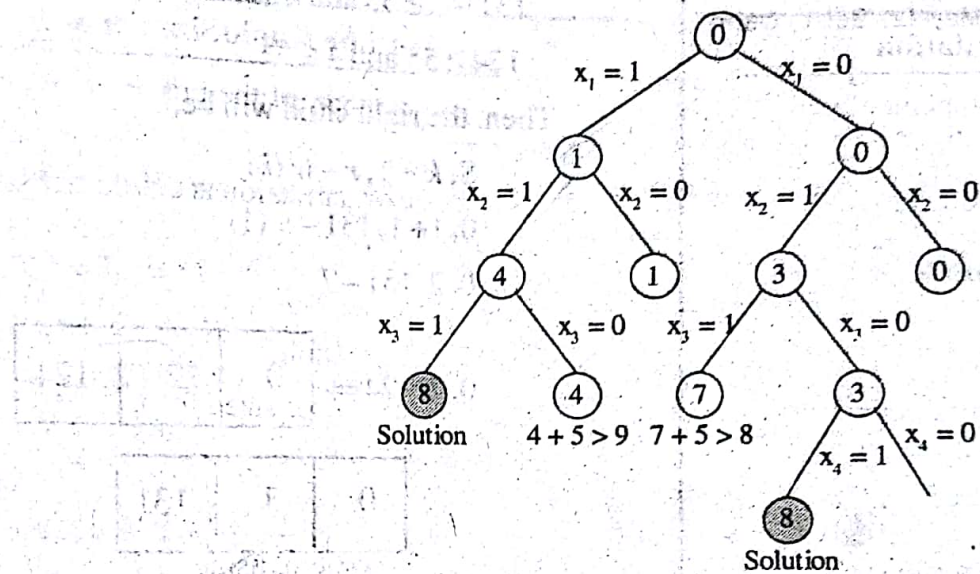
$\rightarrow$ sum of the elements of subset S' is equal to M .

Thus, backtrack one step and continue the search.

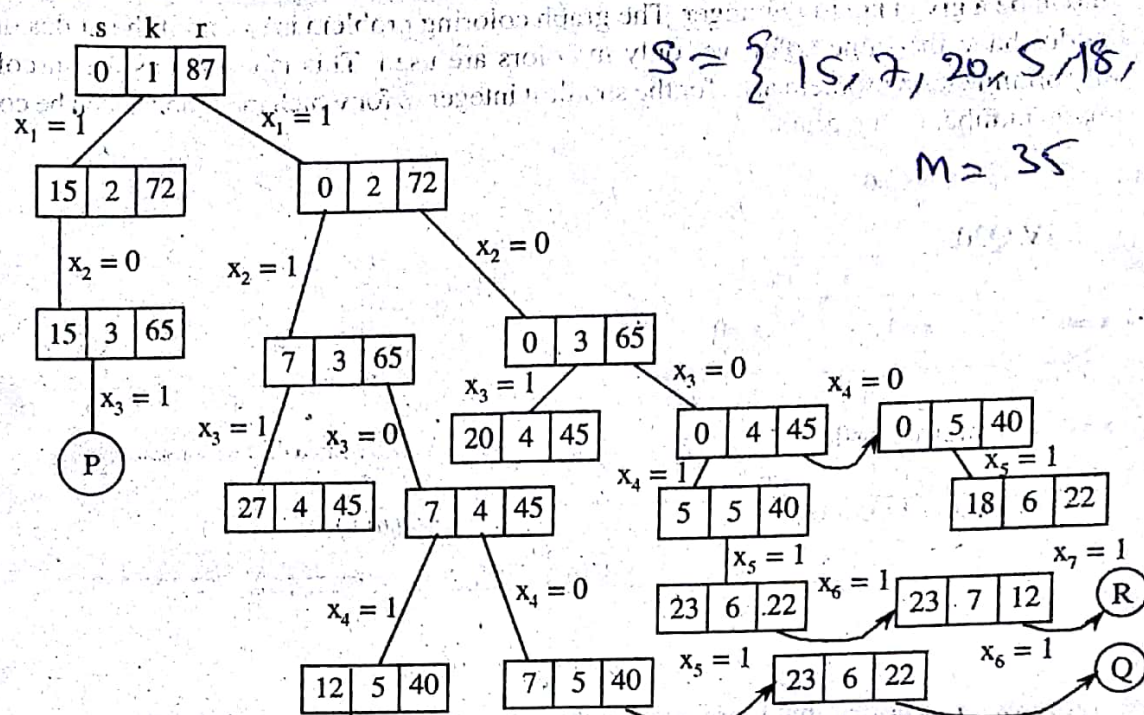
Example

As an example let $S = (1, 3, 4, 5)$ and $M = 8$.

Initially, $S = (1, 3, 4, 5)$ and $M = 8, S' = (\phi)$



Hence, the solutions are $(1, 1, 1, 0)$ and $(0, 1, 0, 1)$.



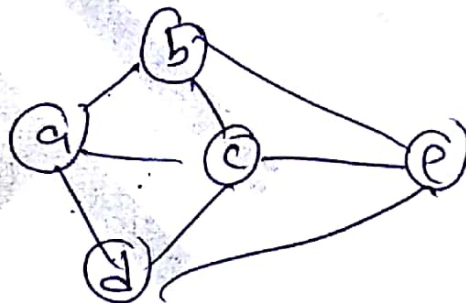
As we can see from the above figure, there are three subsets of ω which sum to m . They are as follows.

1. $(1, 0, 1) \rightarrow P$
i.e., $w_1 + w_3 = 35$
2. $(0, 1, 0, 0, 1, 1) \rightarrow Q$
i.e., $w_2 + w_5 + w_6 = 35$
3. $(0, 0, 0, 1, 1, 0, 1) \rightarrow R$
i.e., $w_4 + w_5 + w_7 = 35$.

Hamiltonian cycles:-

A path $x_0, x_1, \dots, x_{n-1}, x_n$ in the graph $G=(V, E)$ is called a Hamiltonian path, if $V = \{x_0, x_1, \dots, x_n\}$ and $x_i \neq x_j$ for $0 \leq i < j \leq n$. A circuit $x_0, x_1, \dots, x_n$ in a graph $G=(V, E)$ is called a Hamiltonian circuit if $x_0, x_1, \dots, x_n$ is a Hamiltonian path

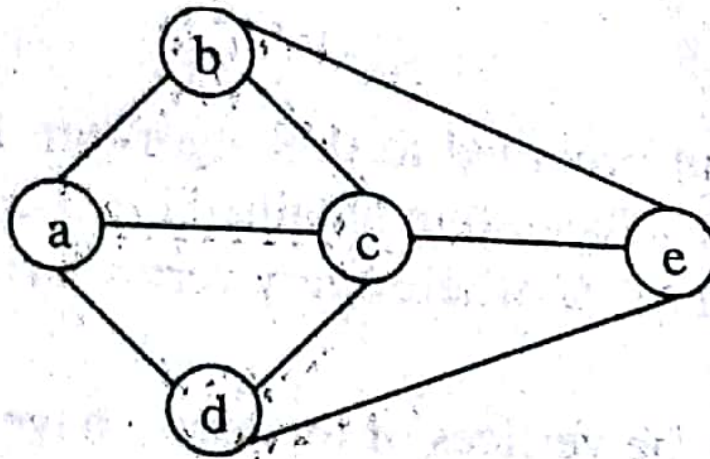
Find Hamiltonian circuit using backtracks



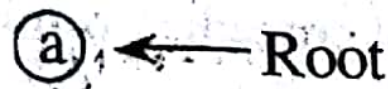
→ a b c e d a

→ a d c e b a

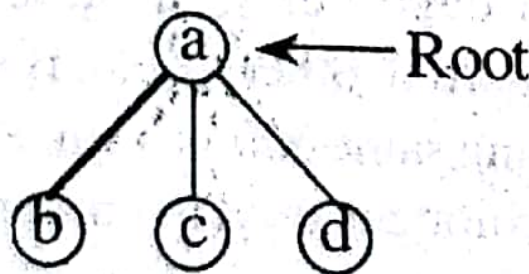
Consider a graph $G = (V, E)$ shown in the following figure, now find a Hamiltonian circuit using backtracking method.



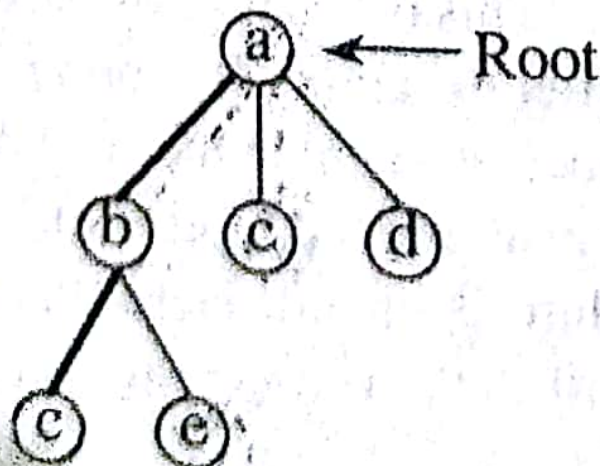
Initially, start the search with vertex 'a', the vertex 'a' becomes the root of our implicit tree.



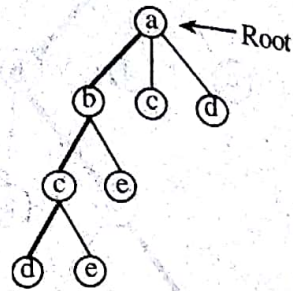
Next choose vertex 'b' adjacent to 'a', as it comes first in lexicographical order (b, c, d).



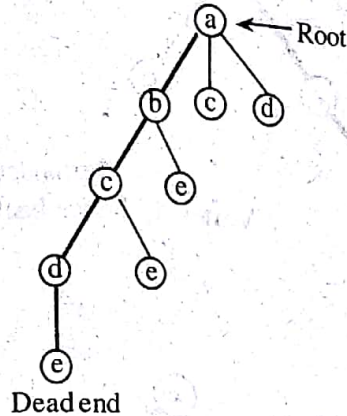
Next vertex 'c' is selected which is adjacent to 'b' and which comes first in lexicographical order (c, e).



Next select vertex 'd' adjacent to 'c' which comes first in lexicographical order (d, e).



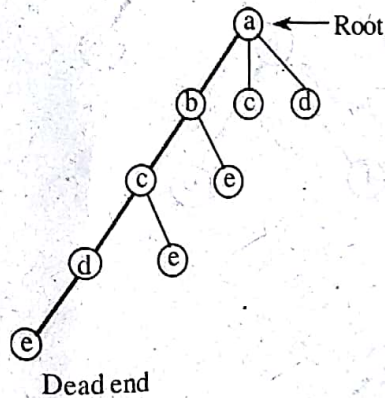
Next vertex 'e' is selected. If vertex 'a' is chosen then the Hamiltonian cycle can not be obtained.



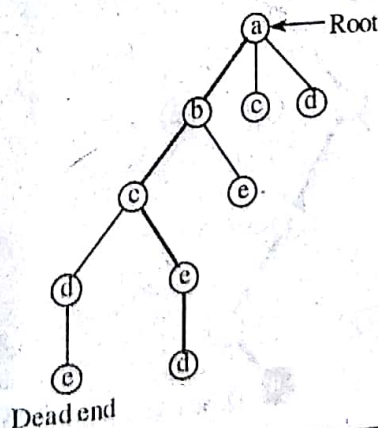
The vertex adjacent to 'e' are b, c, d but they are already visited. Thus, the dead end is obtained. So, backtrack one step and remove the vertex 'e' from the partial solution.

The vertex adjacent to 'd' are e, c, a from which vertex 'e' has already been checked therefore left with the vertex 'a' if left but by choosing vertex 'a' it is not possible to get the Hamiltonian cycle. So, again backtrack one step.

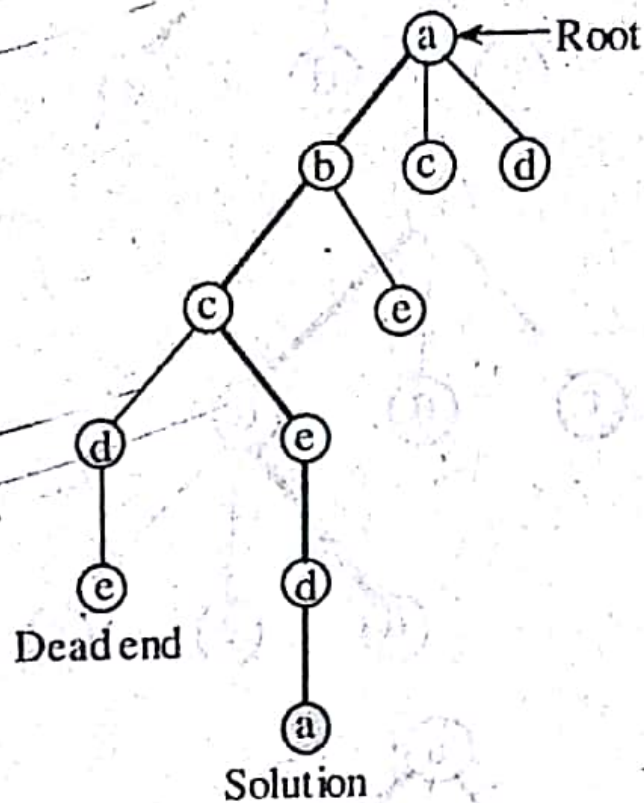
Hence, select the vertex 'e' adjacent to 'c'.



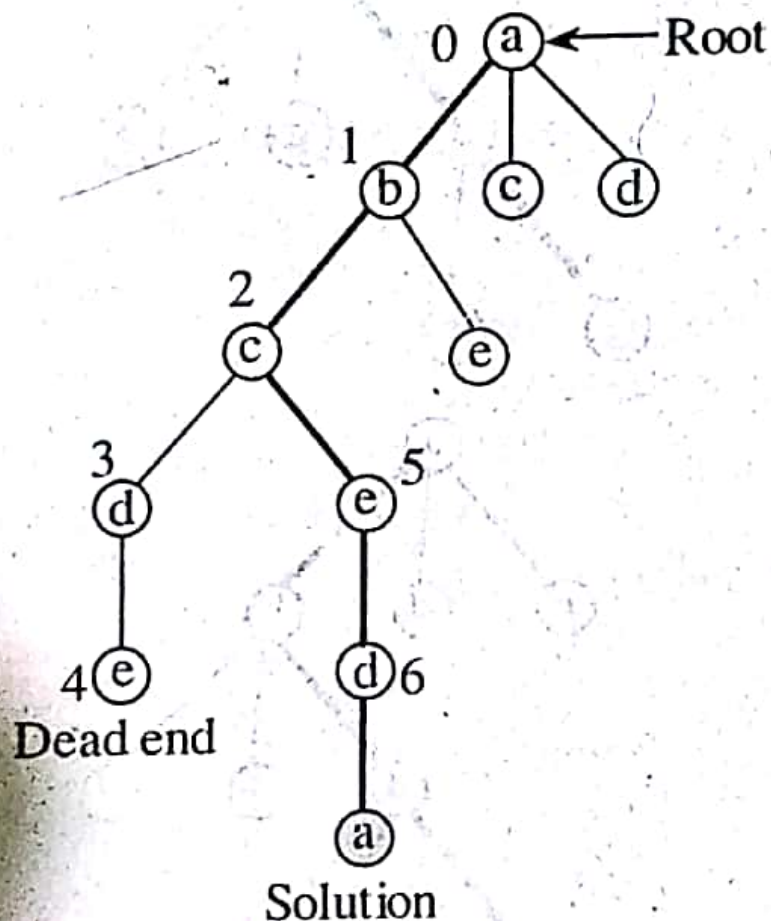
The vertex adjacent to 'e' are (b, c, d) so vertex d is selected.



The vertex adjacent to 'd' are (a, c, e) so vertex 'a' is selected. Hence, the Hamiltonian cycle is obtained as all the vertex other than the start vertex 'a' is visited only once, $a-b-c-e-d-a$.



The final implicit tree for the Hamiltonian circuit is shown in the following figure. The number above each node indicates the order in which these nodes are visited.



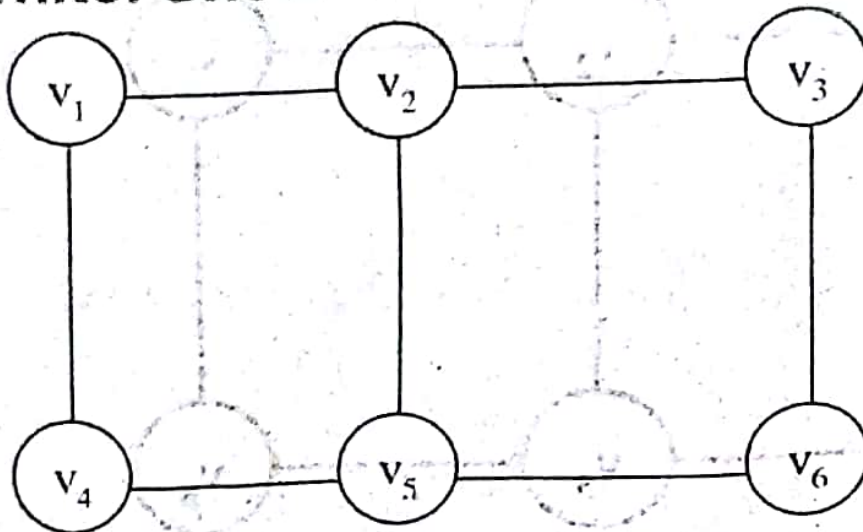


Graph coloring:-

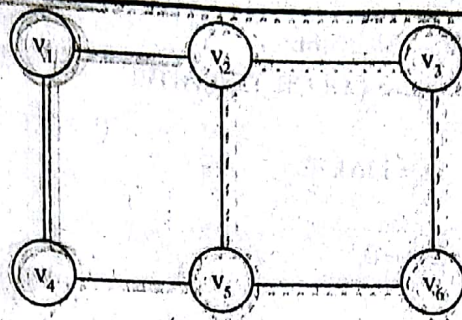
Consider $G(V, E)$ is a undirected graph.

A valid coloring of 'G' is an assignment of colors to the vertices in such a way that each vertex must be assigned with one color and there should be no two adjacent vertices having same color.

Use the backtracking algorithm for the m-coloring problem to find all possible colorings of the graph using the three colors red, green and white. Show the actions step by step.

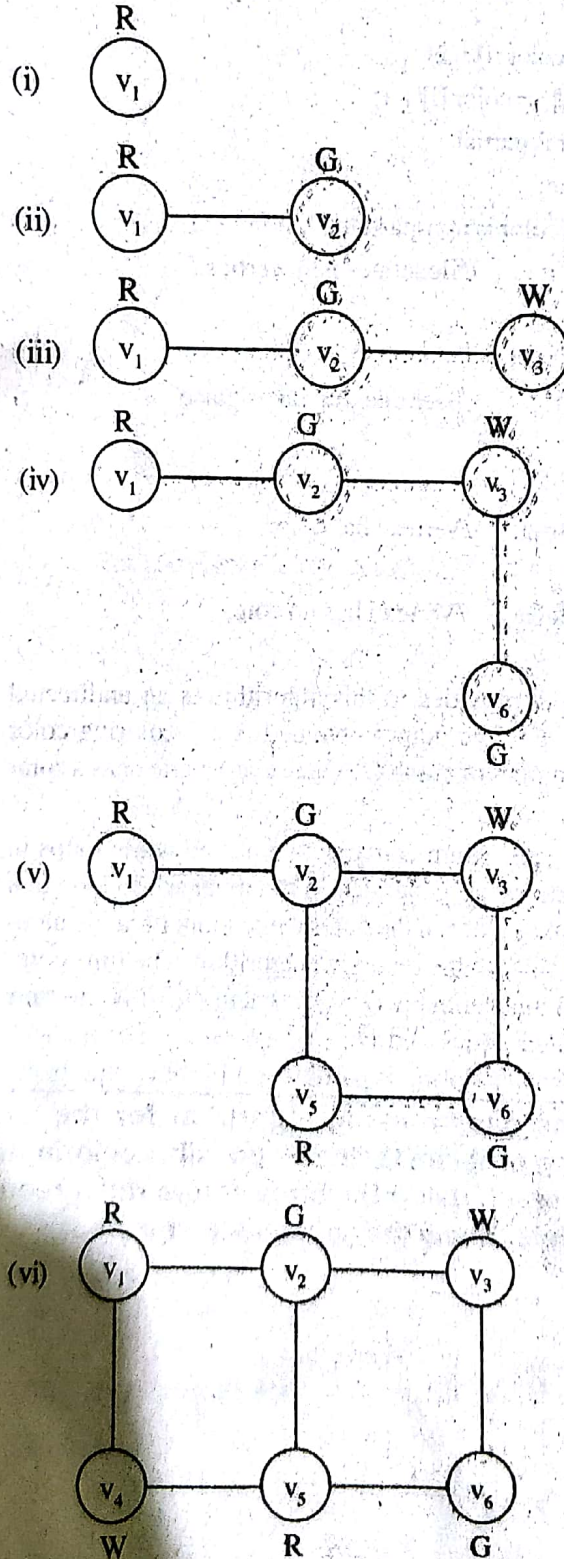


Figure



Graph

Let the colors red, green and white be represented by R , G and W . The step-wise coloring of graph using R , G and W is given below,



Here, backtracking technique is not applied as there is no problem in assigning either of the three colors.

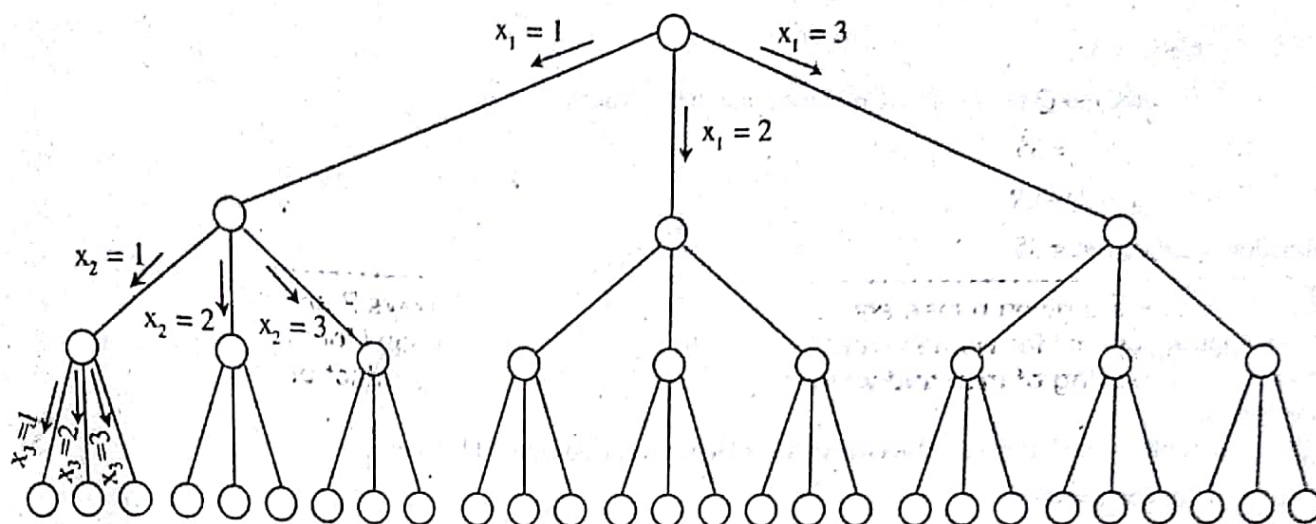
Q29. Draw the portion of the state space tree for m-colorings of a graph.

Answer :

State space tree for m-Coloring when $n = 3, m = 3$

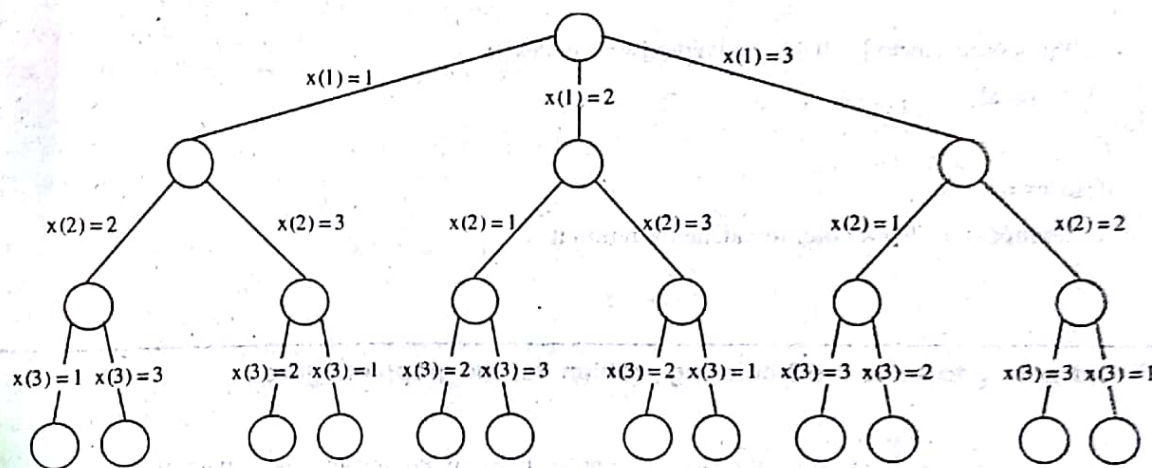
April/May-12, Set-2, Q6(b)

It means, there are 3 vertices in graph and we have to use at most 3 colors, to color the graph.



State space tree contains, all possible moves (some moves leads to solution or may not leads to solution)

The following tree contains only possible solutions.



The possible solutions are 12, within that, 6 solutions are exist with exactly 2 colors. In the above tree, when $x(1) = 1$, then $x(2)$ contain either 2 or 3 but not 1, since two adjacent nodes should not have same color, when $x(2) = 2$, then $x(3)$ contain either 1 or 3. Similarly, we can process remaining nodes.